

TP POO

SOMMAIRE

Exercice 1 :

1.1

1.2

1.3

1.4

1.5

Résultat :

Exercice 2 :

2.1

2.2

2.3

2.4

2.5

2.6

2.7

2.8

Exercice 1 :

1.1

On commence par définir les attributs suivant (Titre, Auteur, Prix)

```
public class livre {  
    // 1. Définir une classe Livre avec les attributs suivants  
    private String titre;  
    private String auteur;  
    private double prix;
```

1.2

Ensuite on définit un constructeur permettant d'initialiser les attributs de la méthode par des valeurs saisies par l'utilisateur

```
// 2. Définir un constructeur permettant d'initialiser les attributs  
public livre(String titre, String auteur, double prix) {  
    this.setTitre(titre);  
    this.setAuteur(auteur);  
    this.setPrix(prix);  
}
```

1.3

Puis on définit à l'aide des propriétés les méthodes d'accès aux différents attributs de la classe :

```
// 3. Définir à l'aide des propriétés les méthodes d'accès  
    public String getTitre() {  
        return titre;  
    }  
    public void setTitre(String titre) {  
        this.titre = titre;  
    }  
    public String getAuteur() {  
        return auteur;  
    }  
    public void setAuteur(String auteur) {  
        this.auteur = auteur;  
    }  
    public double getPrix() {  
        return prix;  
    }  
    public void setPrix(double prix) {  
        this.prix = prix;
```

```
}
```

1.4 On définit ensuite la méthode d'affichage.

La méthode Afficher() est définie pour afficher les informations de l'objet (Titre, Auteur, Prix) sur la console. Elle utilise System.out.println pour concaténer et présenter ces attributs sous forme de chaîne de caractères lisible.

```
// 4. Définir la méthode Afficher()
public void Afficher() {
    System.out.println("Titre: " + this.titre + ", Auteur: " + this.auteur + ", Prix: " + this.prix);
}
```

1.5 Écrire un programme testant la classe Livre :

On commence par afficher livre 1 pour indiquer le début le début du test, puis on simule la saisie d'un utilisateur

```
// --- 1. Livre 1 ---

System.out.println("Livre 1:");

// Simulation des saisies utilisateur
System.out.println("Donner le titre: Programmer en C");
System.out.println("Donner l'auteur: Claude Delannoy");
System.out.println("Donner le prix: 350");
```

Ces lignes affichent simplement des messages pour simuler ce qui se passerait si l'utilisateur entrait ces données. Elles ne créent pas encore l'objet.

On va donc ensuite déclaré un nouvel objet

On crée un nouvel objet 'livre' appelé "livre1" en utilisant le constructeur défini dans la classe `livre` (Point 1.2), qui initialise ses attributs (Titre, Auteur, Prix).

```
// création Livre 1
livre livre1 = new livre("Programmer en C", "Claude Delannoy", 350.0);
```

Ensuite on va faire la vérification des attributs via les getters

```
// vérification affichage des attributs via les getters
System.out.println("Le titre est " + livre1.getTitre());
System.out.println("L'auteur est " + livre1.getAuteur());
System.out.println("Le prix est " + livre1.getPrix());
```

Puis on fait l'affichage complet via la méthode afficher

```
// Affichage final avec la méthode Afficher() (Point 4)
livre1.Afficher();
```

Résultat :

```
Livre 1:
Donner le titre: Programmer en C
Donner l'auteur: Claude Delannoy
Donner le prix: 350
Le titre est Programmer en C
L'auteur est Claude Delannoy
Le prix est 350.0
Titre: Programmer en C, Auteur: Claude Delannoy, Prix: 350.0
```

Exercice 2 :

2.1 On commence par définir les attributs suivant (CIN, Nom, Prénom, Tél.)

```
public class Client {
    // 1. Définir une classe Client avec les attributs suivants
    private String cin;
    private String nom;
    private String prenom;
    private String tel;
```

2.2 On définit ensuite à l'aide des propriétés les méthodes d'accès aux différents attributs de la classe :

```
//2. Définir à l'aide des propriétés les méthodes d'accès aux différents attributs de la classe.
    public String getCin() {
        return cin;
    }
    public void setCin(String cin) {
        this.cin = cin;
    }
    public String getNom() {
        return nom;
    }
    public void setNom(String nom) {
        this.nom = nom;
    }
}
```

```

    public String getPrenom() {
        return prenom;
    }
    public void setPrenom(String prenom) {
        this.prenom = prenom;
    }
    public String getTel() {
        return tel;
    }
    public void setTel(String tel) {
        this.tel = tel;
    }
}

```

2.3 On définit les constructeurs permettant d'initialiser tous les attributs

```

public Client(String cin, String nom, String prenom, String tel) {
    this.setCin(cin);
    this.setNom(nom);
    this.setPrenom(prenom);
    this.setTel(tel);
}

```

2.4 On définit la méthode d'affichage

```

// 4. Définir la méthode Afficher()
public void Afficher() {
    System.out.println("Propriétaire du compte :");
    System.out.println("CIN: " + this.cin);
    System.out.println("NOM: " + this.nom);
    System.out.println("Prénom: " + this.prenom);
    System.out.println("Tél: " + this.tel);
}
}

```

2.5

```
public class Compte {  
    // 5. Code qui est incrémenté lors de sa création (variable de classe stat  
    private static int dernierCode = 0;  
  
    // Attributs  
    private final int numeroDeCompte; // Le code unique et incrémenté  
    private double solde;  
    private final Client proprietaire; // Le propriétaire (un objet Client)
```

2.6

```
    // 6. Définir les méthodes d'accès |  
    public int getNumeroDeCompte() {  
        return numeroDeCompte;  
    }  
  
    public double getSolde() {  
        return solde;  
    }  
  
    public Client getProprietaire() {  
        return proprietaire;  
    }  
}
```

2.7

```
    // 7. Définir un constructeur en indiquant son propriétaire  
    public Compte(Client proprietaire) {  
        Compte.dernierCode++;  
        this.numeroDeCompte = Compte.dernierCode; // Attribution du code  
  
        this.solde = 0.0; // Solde initial à zéro  
        this.proprietaire = proprietaire;  
    }  
}
```

2.8

```
// --- 8. Méthodes de transaction ---
```

```
// Une méthode permettant de Crediter() le compte, prenant une somme en para
public boolean Crediter(double montant) {
    if (montant > 0) {
        this.solde += montant;
        return true;
    }
    return false;
}

// Une méthode permettant de Debiter() le compte, prenant une somme en para
public boolean Debiter(double montant) {
    if (montant > 0 && this.solde >= montant) {
        this.solde -= montant;
        return true;
    }
    return false;
}
```

```
// Créditant le compte en cours et débitant le compte passé en paramètres.
public boolean Crediter(Compte compteSource, double montant) {
    // Débit de la source
    if (compteSource.Debiter(montant)) {
        // Crédit du compte en cours (le 'this' implicite)
        this.Crediter(montant);
        return true;
    }
    return false;
}
```

```
// Une méthode permettant de Debiter() le compte, prenant une somme et un co
// Débitant le compte en cours et créditant le compte passé en paramètres.
public boolean Debiter(Compte compteDestination, double montant) {
    // Débit du compte en cours
    if (this.Debiter(montant)) {
        // Crédit de la destination
        compteDestination.Crediter(montant);
        return true;
    }
    return false;
}
```

```
// Une méthode qui permet d'afficher le résumé d'un compte.
public void Resumer() {
    System.out.println("\n*****");
    System.out.println("Détails du compte:");
    System.out.println("Numéro de Compte: " + this.numeroDeCompte);
    System.out.println("Solde de compte: " + this.solde);
    // Utilisation de la méthode Afficher() de l'objet Client
    this.proprietaire.Afficher();
    System.out.println("*****");
}
```